

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

**COURSE CODE: CSA14
DESIGN**

COURSE NAME: COMPILER

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)*

Assessment Tool Weight-age: 40%

QUESTIONS: 5

Total Marks:50

Annexure A

SIMULATION TASK

Code of Conduct:

I ____M.Madhu Rushwika(192371066)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.Madhu Rushwika

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts

Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Annexure A

SIMULATION TASK

QUESTIONS

1. Given the three-address code sequence:

```
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
```

Simulate the **peephole optimization** process on this sequence. Produce the optimized instruction sequence and justify each optimization step applied, including algebraic simplification, constant folding, and dead code elimination.

2. Partition the following intermediate code into **basic blocks** and construct the corresponding **control flow graph**. Identify all leader statements and mention the rules used to identify them:

```
(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6
(4) z = y - d
(5) goto 7
(6) z = y + d
(7) w = z * 2
```

3. Construct the **DAG** for the following basic block:

```
p = a + b
q = a + b
r = p * q
s = a + b
t = r + s
```

Using the DAG, identify all **common sub-expressions**. Then simulate a simple code generator to produce optimized target code and explain the register allocation decisions at each step.

4. For a target machine with two registers **R0** and **R1**, simulate the working of a **simple code generator** for the code:

```
t1 = a + b
t2 = c * d
t3 = t1 - t2
```

t4 = e + f
t5 = t3 / t4

Show the **register descriptor** and **address descriptor** after each instruction is processed.

5. For a **RISC-style target machine** with instructions **LOAD, STORE, ADD, SUB, MUL, MOV**, simulate target code generation for the expression:

$(a - b) * (c + d) - e$

Show the complete sequence of target instructions generated. Also explain the **runtime storage management** and **stack frame layout** used during the computation.

Given the three-address code sequence:

t1 = a + b,
t2 = t1 + 0,
t3 = t2 * 1,
if t3 == 0 goto L1,
t4 = t3 + t3

1. Peephole Optimization

Given:

t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y

Step 1: Algebraic simplification

- $x * 1 \rightarrow x$
- $t2 = x + y$
- $t3 = t2 - 0 \rightarrow t2$
- $t4 = t3 * 0 \rightarrow 0$

So:

t1 = x
t2 = x + y
t3 = t2
t4 = 0
if 0 != 0 goto L1
t5 = y + y

Step 2: Constant folding

Since:

$0 \neq 0$

is always **false**, the conditional jump can never occur.

Therefore:

if 0 != 0 goto L1

is removed.

Step 3: Dead code elimination

After the condition is removed:

- t4 is never used.
- t3 is only used to compute t4, which is also useless.
- Therefore t3 and t4 are dead.
- t1 is only used by t2, so it can be substituted directly.

Optimized sequence

t2 = x + y

t5 = y + y

Optimizations applied

Original	Optimization	Result
t1 = x * 1	Algebraic simplification	t1 = x
t3 = t2 - 0	Algebraic simplification	t3 = t2
t4 = t3 * 0	Algebraic simplification	t4 = 0
if t4 != 0	Constant folding	Always false
t3, t4	Dead code elimination	Removed
t1	Copy propagation	Eliminated

Final optimized code:

t2 = x + y

t5 = y + y

2. Basic Blocks and Control Flow Graph

Intermediate code:

- (1) x = a + b
- (2) y = x * c
- (3) if y > 50 goto 6
- (4) z = y - d
- (5) goto 7
- (6) z = y + d
- (7) w = z * 2

Leader identification rules

A statement is a **leader** if:

1. The first statement of the program is a leader.
2. Any statement that is the target of a conditional/unconditional jump is a leader.
3. Any statement immediately following a jump statement is a leader.

Leaders

- 1 $\rightarrow$ first statement
- 4 $\rightarrow$ immediately follows conditional jump at 3
- 6 $\rightarrow$ target of goto 6
- 7 $\rightarrow$ target of goto 7

Therefore:

Leaders = {1, 4, 6, 7}

Basic blocks

B1

- (1) $x = a + b$
- (2) $y = x * c$
- (3) if $y > 50$ goto 6

B2

- (4) $z = y - d$
- (5) goto 7

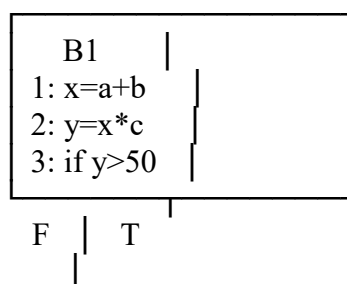
B3

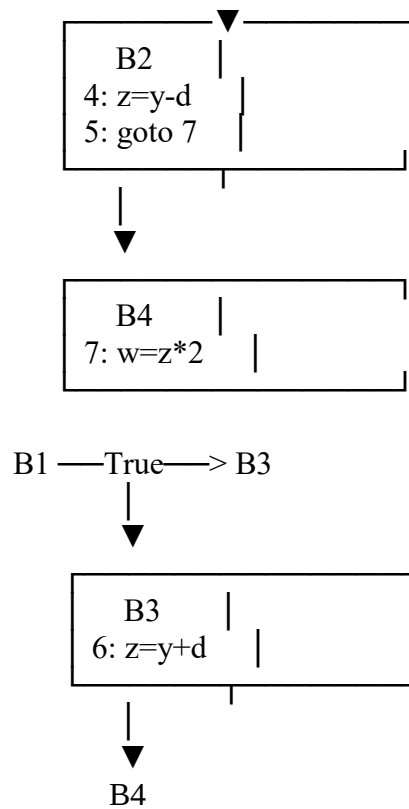
- (6) $z = y + d$

B4

- (7) $w = z * 2$

Control Flow Graph





CFG edges

$B1 \rightarrow B2 \quad (y \leq 50)$
 $B1 \rightarrow B3 \quad (y > 50)$
 $B2 \rightarrow B4$
 $B3 \rightarrow B4$

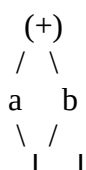
3. DAG Construction and Common Subexpressions

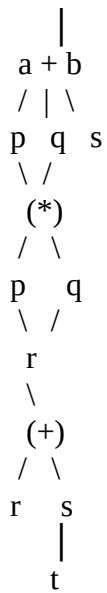
Given:

$p = a + b$
 $q = a + b$
 $r = p * q$
 $s = a + b$
 $t = r + s$

DAG

The expression $a + b$ occurs three times.





More formally:

$N1 = a$
 $N2 = b$
 $N3 = +(N1, N2) \rightarrow a+b$

$p \rightarrow N3$
 $q \rightarrow N3$
 $s \rightarrow N3$

$N4 = *(N3, N3) \rightarrow p*q$
 $r \rightarrow N4$

$N5 = +(N4, N3) \rightarrow r+s$
 $t \rightarrow N5$

Common sub-expression

$a + b$

appears in:

$p = a + b$
 $q = a + b$
 $s = a + b$

Thus, calculate it only once.

Optimized three-address code

$p = a + b$
 $r = p * p$
 $t = r + p$

q and s are unnecessary because both contain the same value as p.

Simple code generation

Assume registers R0 and R1.

```
LOAD R0, a
ADD  R0, b    ; R0 = a+b
```

```
MUL  R0, R0    ; R0 = p*p
```

```
LOAD R1, a
ADD  R1, b    ; R1 = p
```

```
ADD  R0, R1    ; R0 = r+p
```

```
STORE t, R0
```

A better generator can preserve p:

```
LOAD R0, a
ADD  R0, b    ; R0 = p
```

```
MOV  R1, R0    ; R1 = p
MUL  R1, R1    ; R1 = r
```

```
ADD  R1, R0    ; R1 = t
STORE t, R1
```

Register decisions

Step	R0	R1
Load a	a	—
ADD b	p = a+b	—
MOV R1,R0	p	p
MUL R1,R1	p	r
ADD R1,R0	p	t
STORE t,R1	p	t

4. Code Generation with Two Registers

Given:

t1 = a + b

t2 = c * d

$t3 = t1 - t2$

$t4 = e + f$

$t5 = t3 / t4$

Assume registers R0 and R1.

Register and address descriptors

Initially:

Register Descriptor:

R0 $\rightarrow$ empty

R1 $\rightarrow$ empty

Address Descriptor:

a $\rightarrow$ memory

b $\rightarrow$ memory

c $\rightarrow$ memory

d $\rightarrow$ memory

e $\rightarrow$ memory

f $\rightarrow$ memory

Instruction 1: $t1 = a + b$

LOAD R0, a

ADD R0, b

Descriptors:

R0 $\rightarrow$ t1

R1 $\rightarrow$ empty

t1 $\rightarrow$ R0

a $\rightarrow$ memory

b $\rightarrow$ memory

Instruction 2: $t2 = c * d$

LOAD R1, c

MUL R1, d

Descriptors:

R0 $\rightarrow$ t1

R1 $\rightarrow$ t2

t1 $\rightarrow$ R0

t2 $\rightarrow$ R1

Instruction 3: $t3 = t1 - t2$

SUB R0, R1

Now:

$R0 \rightarrow t3$

$R1 \rightarrow t2$

Address descriptor:

$t3 \rightarrow R0$

$t2 \rightarrow R1$

$t2$ is dead after this point, so $R1$ can be reused.

Instruction 4: $t4 = e + f$

LOAD R1, e

ADD R1, f

Descriptors:

$R0 \rightarrow t3$

$R1 \rightarrow t4$

$t3 \rightarrow R0$

$t4 \rightarrow R1$

Instruction 5: $t5 = t3 / t4$

DIV R0, R1

Final:

$R0 \rightarrow t5$

$R1 \rightarrow t4$

Address descriptor:

$t5 \rightarrow R0$

$t4 \rightarrow R1$

Complete target code

LOAD R0, a

ADD R0, b

LOAD R1, c

MUL R1, d

SUB R0, R1

LOAD R1, e
ADD R1, f

DIV R0, R1

5. RISC Target Code Generation

Expression:

$(a - b) * (c + d) - e$

Assume:

- R0–R3 are general-purpose registers.
- LOAD loads a value from memory.
- STORE stores a register value to memory.
- ADD, SUB, MUL perform arithmetic.
- MOV copies a register.

Step 1: Compute (a-b)

LOAD R0, a
LOAD R1, b
SUB R0, R1

Now:

$R0 = a - b$

Step 2: Compute (c+d)

LOAD R2, c
LOAD R3, d
ADD R2, R3

Now:

$R2 = c + d$

Step 3: Multiply

MUL R0, R2

Now:

$R0 = (a - b) * (c + d)$

Step 4: Load e

LOAD R1, e

Step 5: Subtract e

SUB R0, R1

Final result:

$R0 = (a-b)*(c+d)-e$

Complete target code

LOAD R0, a
LOAD R1, b
SUB R0, R1

LOAD R2, c
LOAD R3, d
ADD R2, R3

MUL R0, R2

LOAD R1, e
SUB R0, R1

STORE result, R0

Runtime storage management

During execution, variables are generally stored in memory while intermediate results are held in registers.

A simple stack frame can be represented as:

Higher Address

Return Address	
Saved Registers	
Local Variables a, b, c, d, e	
Temporaries temp1 = a-b temp2 = c+d result	

Lower Address

The compiler tries to keep frequently used intermediate values in registers to reduce memory access. If registers become insufficient, a value may be **spilled** to the stack.

Additional Given Three-Address Code

You also provided:

```
t1 = a + b
t2 = t1 + 0
t3 = t2 * 1
if t3 == 0 goto L1
t4 = t3 + t3
```

Optimization

1. Algebraic simplification

```
t2 = t1 + 0
```

becomes:

```
t2 = t1
```

And:

```
t3 = t2 * 1
```

becomes:

```
t3 = t2
```

Using copy propagation:

```
t3 = a + b
```

2. Conditional expression

```
if t3 == 0 goto L1
```

cannot be constant-folded because $a + b$ is not known at compile time.

Therefore the condition must remain.

3. Final optimized TAC

```
t1 = a + b
if t1 == 0 goto L1
t4 = t1 + t1
```

Or, with temporary elimination:

```
t1 = a + b
if t1 == 0 goto L1
t4 = t1 + t1
```

Key point: Unlike the first problem, the branch here **cannot be removed**, because a and b are runtime values.

RUBRICS FOR CALCULATION

Simulation tools

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

